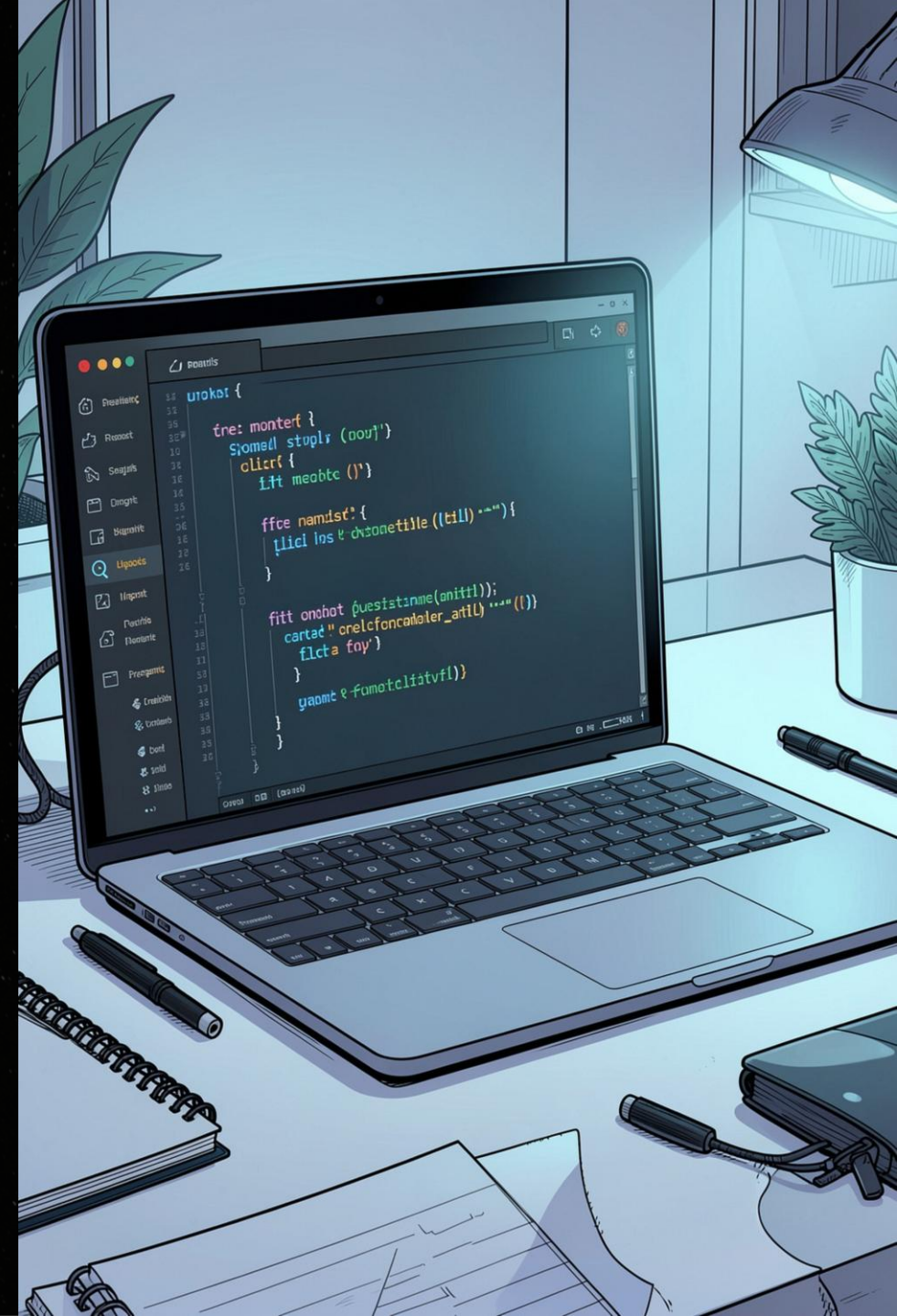
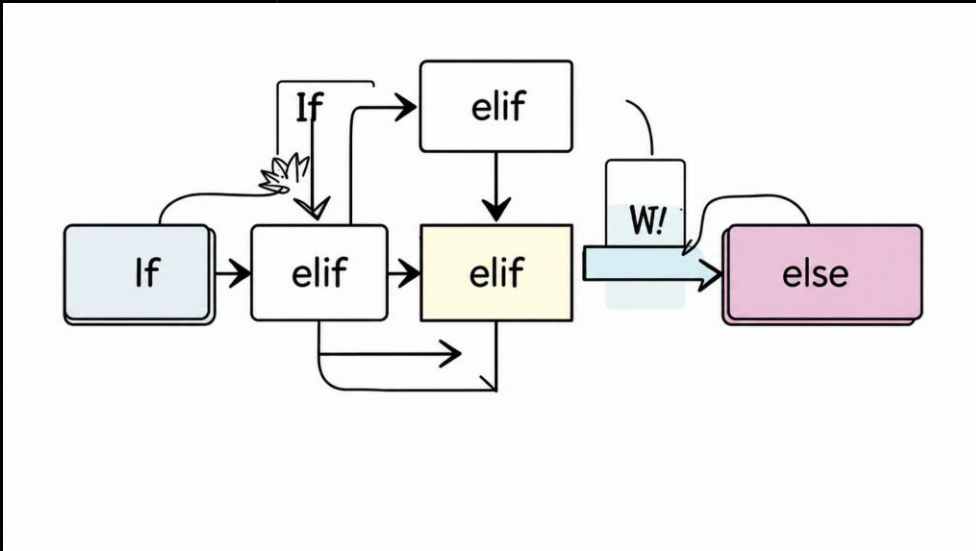


Python: From Foundational Logic to Machine Learning

- **Presenters:** Sanvi, Shivansh and Shayan
- **Batch:** AIML Training (from CSE)
- **Date:** 27/07/2026



Flow Control: If, Loops, and Flow Modifiers



Conditional Branching

if / elif / else — guard logic, nested vs flat style

Loops

- for: iterate sequences.
- while: entry level loops.
- do-while: exit level loops.

Modifiers

break (exit), continue (next iteration)

Fundamentals Of Python



List

Mutable, ordered, undexable



Set

Unordered, unique — fast membership checks



Tuple

Immutable, ordered — use for fixed records



Dict

Key → value mapping — fast lookup by key

System Interactions: File Handling & Modular Code(File I/O)

01

File Modes

- `"r"` opens a file for read only, `"w"` writes and overwrites existing content, `"a"` appends new content to the end of a file, and `"x"`
- creates a new file and fails if it already exists.

02

File handling details

- **File Operations:**
- `open()`, `close()`, `read()`, `readline()`, `readlines()`, and `write()` support the core workflow for working with files. Use `open()` to access a file, `read()` to load the full contents, `readline()` and `readlines()` to process line by line, `write()` to write data, and `close()` to release resources when not using a context manager.



Numerical Analysis & Visualization



NumPy

ndarray, vectorized ops, linear algebra, performance



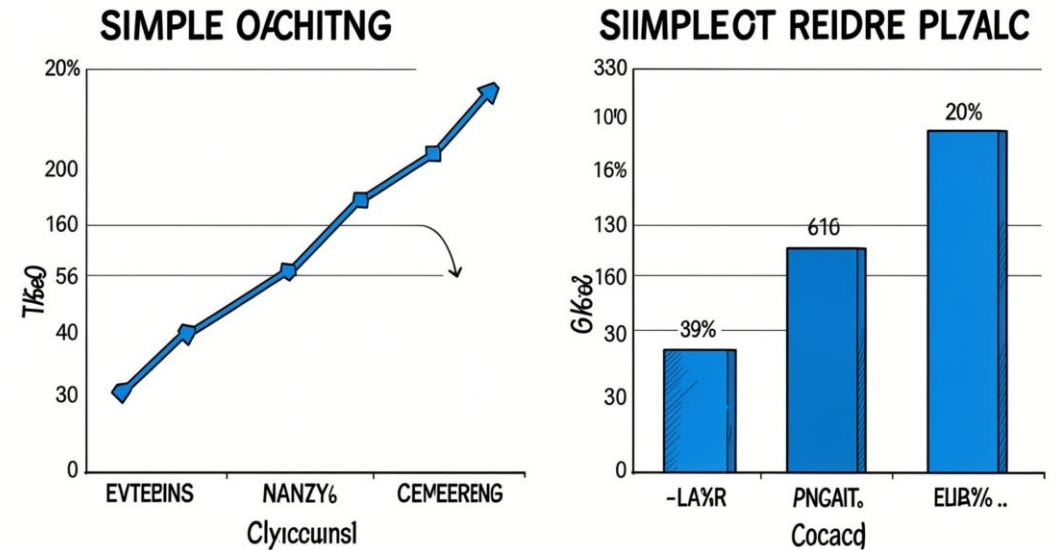
Pandas

Series/DataFrame • cleaning, groupby, merge, pivot



Matplotlib

plots, customization, exploratory visuals



Use visual inspection + summary stats to find trends & outliers

NumPy: The Power of Numerical Arrays

NumPy (Numerical Python) is a foundational library for scientific computing, essential for mathematical calculations, data analysis, machine learning, and AI.

Performance

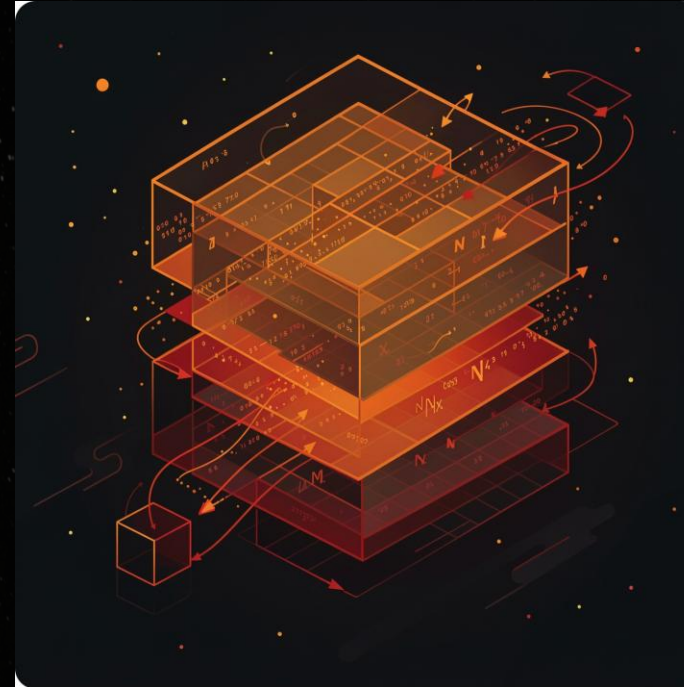
Significantly faster than Python lists for numerical operations, ideal for large datasets.

Efficiency

Uses less memory for storing numerical data, making it suitable for memory-intensive tasks.

Advanced Math

Offers extensive mathematical functions (ufuncs) for complex computations like linear algebra.



At its core, NumPy introduces the **ndarray** (n-dimensional array), a powerful data structure for efficient handling of numerical data, from 1D vectors to multi-dimensional matrices.

Installation:

```
pip install numpy
```

Import:

```
import numpy as np
```

NumPy: Essential Functions and Universal Functions (ufuncs)

NumPy provides a vast array of functions to efficiently manipulate arrays. Among these, **Universal Functions (ufuncs)** are pivotal for performing fast, element-wise operations on arrays, significantly enhancing computational speed and ease of use.



Element-wise Operations

Ufuncs apply mathematical operations like addition, subtraction, or sine to every element of an array independently, without requiring explicit loops.

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([10, 20, 30])

np.add(a, b)      # [11 22 33]
np.multiply(a, b) # [10 40 90]
```



Broadcasting Support

They automatically handle operations between arrays of different but compatible shapes, simplifying code and reducing memory usage.

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])
b = np.array([10, 20, 30])

np.add(a, b)
# [[11 22 33]
#  [14 25 36]]
```

These functions are crucial for scientific computing, enabling complex calculations with minimal code and maximum efficiency.



UFunctions

Leverage C implementations under the hood, ufuncs are dramatically faster than Python's built-in operations for large datasets, making them ideal for performance-critical tasks.

```
# Python loop
result = []
for x, y in zip(a, b):
    result.append(x + y)

# NumPy ufunc
result = np.add(a, b)
```

Practical examples:

- `np.sqrt([1, 4, 9, 16])` → `[1. 2. 3. 4.]`
- `np.sin([0, np.pi/2, np.pi])` → `[0. 1. 0.]`
- `np.maximum([1, 5, 2], [3, 4, 1])` → `[3 5 2]`



Extensive Library

NumPy offers hundreds of ufuncs for mathematical (`np.add`, `np.sqrt`), trigonometric (`np.sin`, `np.cos`), comparison (`np.maximum`), and bitwise operations.

Real-world uses include image processing, signal analysis, feature scaling, and fast thresholding in data pipelines.

Pandas: Data Manipulation & Analysis

Pandas is a foundational library for data science in Python, providing powerful, flexible, and easy-to-use data structures for working with tabular and time-series data. It has the combining power of Excel, SQL, and Python for efficient data handling.



What is Pandas?

Python's powerhouse for structured data, simplifying tasks that would be complex with standard Python lists or NumPy arrays.



Series (1D Data)

A one-dimensional labeled array capable of holding any data type. Conceptually, it's like a single column in a spreadsheet or a SQL table.



DataFrame (2D Data)

A two-dimensional labeled data structure with columns of potentially different types, much like a spreadsheet or a SQL table.

Installing and Importing Pandas

Use pip to install Pandas, then import it with the common alias `pd`.

```
# Install Pandas
pip install pandas
```

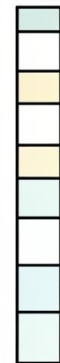
```
# Basic import
import pandas as pd
```

```
# Check version
pd.__version__
```

PANDAS DATA STRUCTURES

Praterfire	KFA	K50	Ser/6
1 '90 tnt2	49	29	16-
1 180	40	38	40
1. Dan19	36	38	\$*1
Mark.S0	31	33	\$*3
Marr.S18	31	33	\$15
Stamr.20	00		\$*3
SITO	00		

Series



Pandas: Essential Data Operations

Pandas provides a comprehensive toolkit for common data manipulation tasks, making data preparation and analysis efficient and intuitive. Let's explore some key functionalities:



Data Input/Output

Easily load data from various sources like CSV, Excel, SQL databases, and JSON files into DataFrames.

- Reading `csv` and `excel` files
- Writing back to files



Data Cleaning & Sorting

Handle missing values, duplicates, and ensure data consistency. Arrange data in meaningful orders.

- Data Cleaning (e.g., handling NaNs)
- Sorting Data by values or index



Aggregation (GroupBy)

Perform powerful aggregation operations to summarize data based on one or more keys.

- `GroupBy` for summarization
- Applying aggregate functions



Reshaping Data (Pivot)

Transform data between "long" and "wide" formats for different analytical needs, similar to spreadsheet pivot tables.

- `Pivot Tables` for summarization
- Stack and Unstack operations



Data Selection & Filtering

Access and isolate specific subsets of data based on labels, integer location, or conditional logic.

- Selecting Data by columns or rows
- Filtering Data with conditions



Data Transformation

Modify existing data or create new derived features to enrich your dataset for analysis.

- Add new Columns and rows
- Apply functions to data



Combining DataFrames

Integrate data from multiple DataFrames using various joining and concatenation methods.

- Merge DataFrames (joins)
- Concat DataFrames

Matplotlib: Data Visualization Essentials

Matplotlib is Python's cornerstone library for creating static, interactive, and animated visualizations. It transforms numerical data into insightful graphs and charts, making complex datasets easily understandable.



Understand Data Quickly

Visualize trends, patterns, and outliers at a glance.



Compare & Analyze Values

Easily compare different data points and relationships.



Effective Reporting

Create professional, publication-quality figures.

Getting Started:

```
# Installation
pip install matplotlib

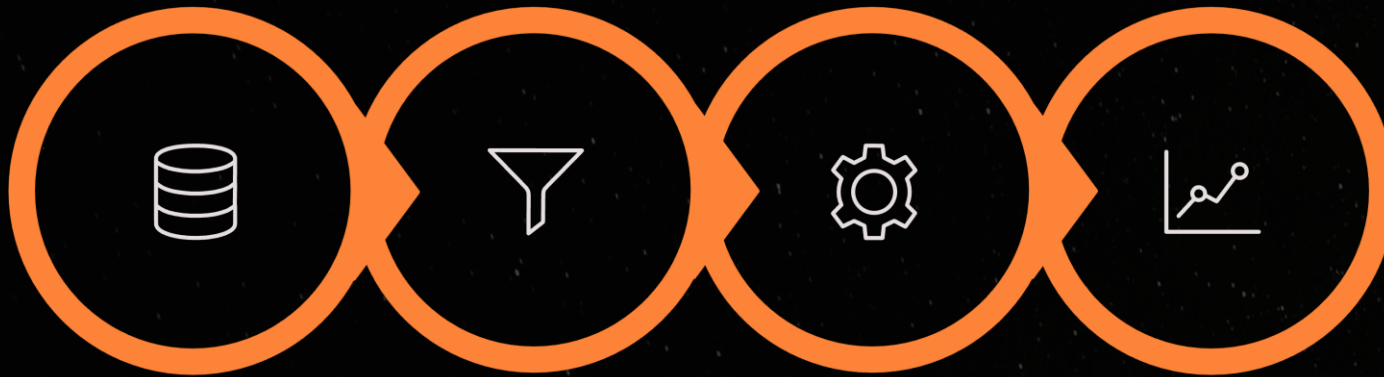
# Import
import matplotlib.pyplot as plt
```

The `pyplot` module provides a MATLAB-like interface for plotting. It's the most commonly used part of Matplotlib.

Basic Plotting Cycle:

- `plt.plot()`, `plt.bar()`, `plt.hist()`, `plt.pie()`, `plt.scatter()` to create the chart.
- `plt.title()`, `plt.xlabel()`, `plt.ylabel()` for context.
- `plt.show()` to display the visualization.

The Machine Learning Pipeline



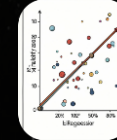
Load data

Preprocess

Train model

Evaluate

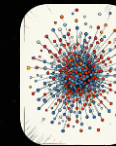
- Always split (train/test)
- Scale features when needed



Regression

LinearRegression — predict continuous data

LogisticalRegression — predict category



KNN

K-Nearest Neighbor

K = number of nearest neighbor



Decision Tree

It is a flowchart like algorithm, which ask series of questions, based on answers it reaches the final decision.

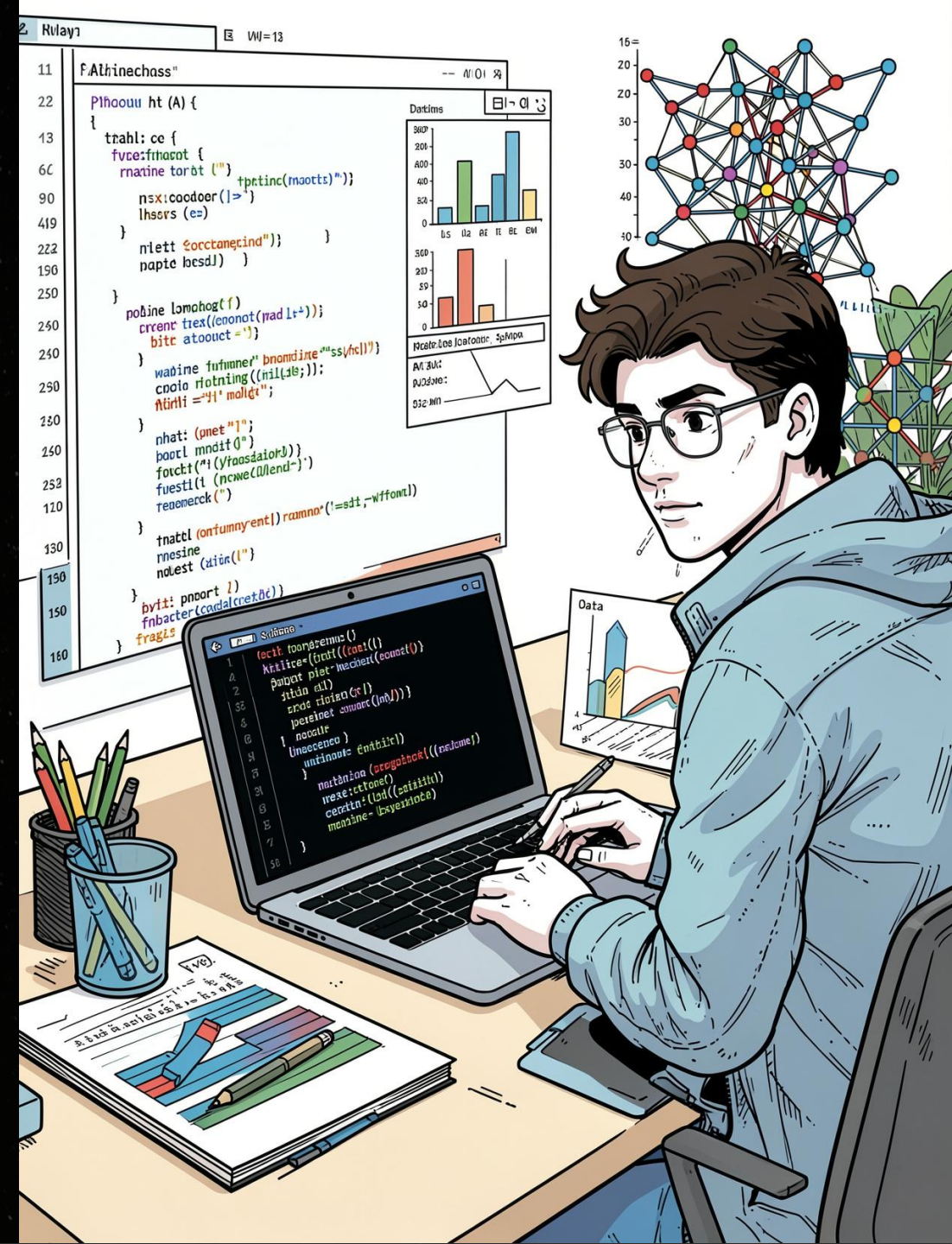
Customer Segmentation, Customer Type Prediction & Country Growth Prediction

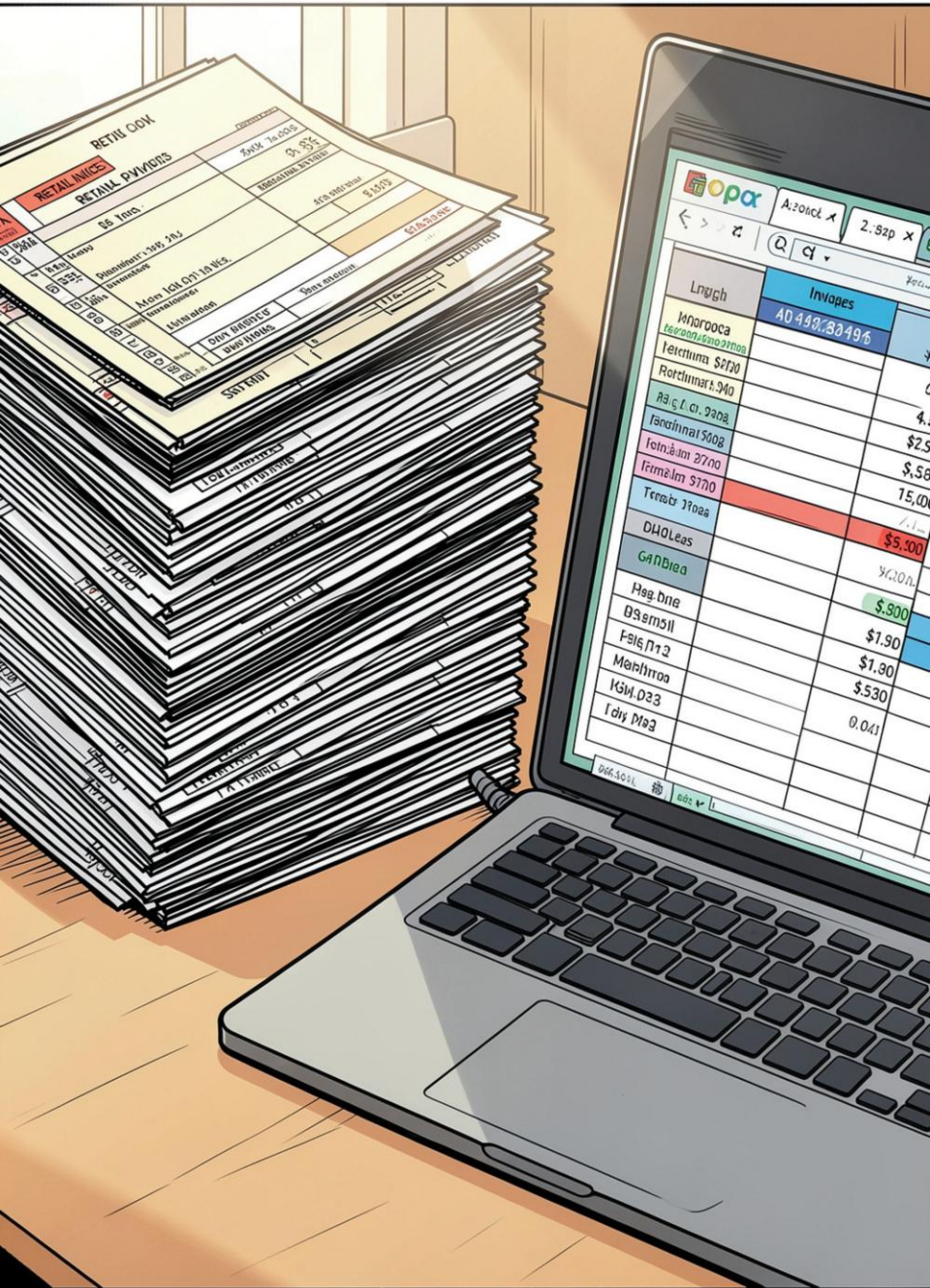


10-Days AIML Training — Key Skills

- Python fundamentals
list, tuple, set, dictionary
- Data handling
NumPy, Pandas — cleaning & transforms
- Visualization
Matplotlib — exploratory plots
- Supervised learning
Linear, Logistic, Random Forest
- Unsupervised learning
KMeans clustering

📁 This project applies these concepts together on a real-world retail dataset.





Project Overview

Problem: Raw transaction data – no customer insight, unclear growth focus.

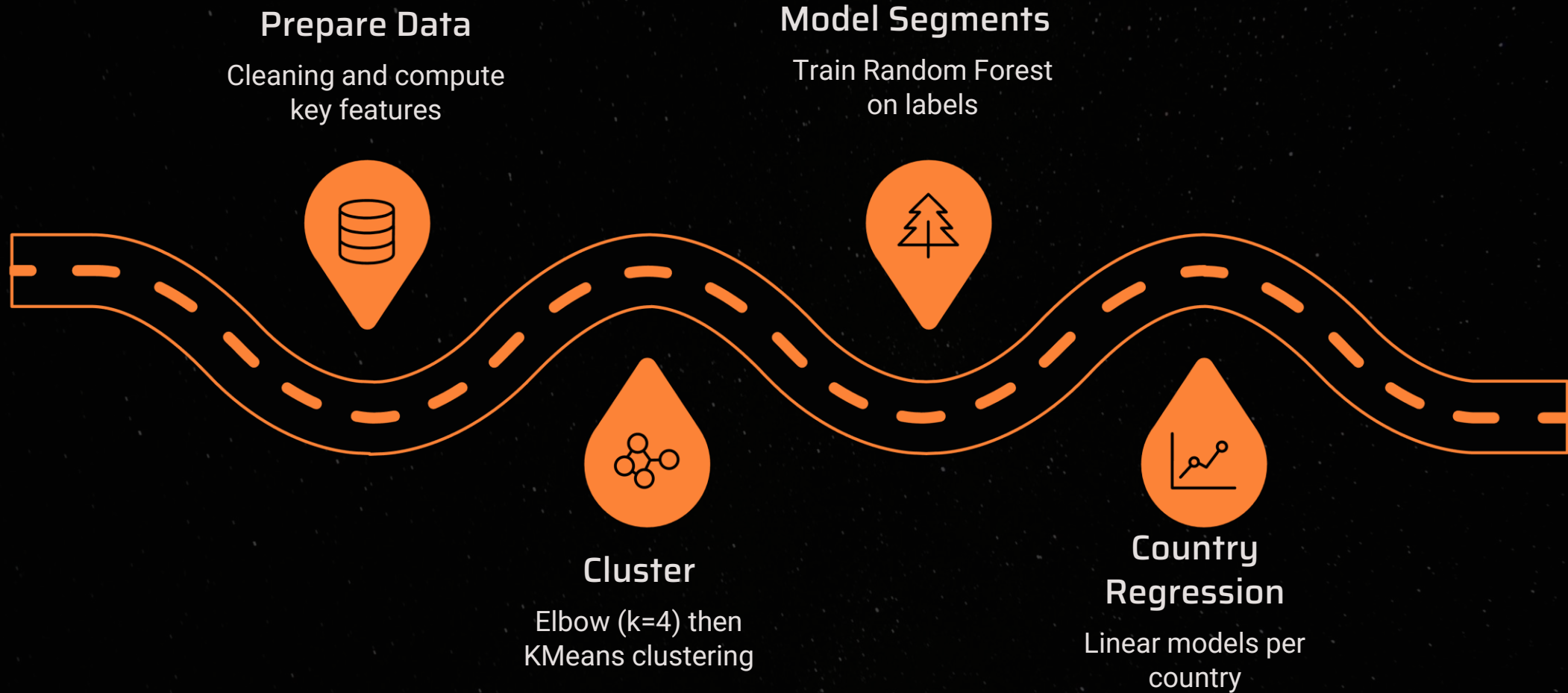
Dataset

- ~540,000 UK online retail transactions (Dec 2010–Dec 2011)
- 8 columns: InvoiceNo, StockCode, Description, Quantity
- InvoiceDate, UnitPrice, CustomerID, Country

Three objectives

1. Segment customers (KMeans)
2. Predict customer type for new customers (Random Forest)
3. Predict country sales growth (Linear Regression)

Methodology — High Level Flow

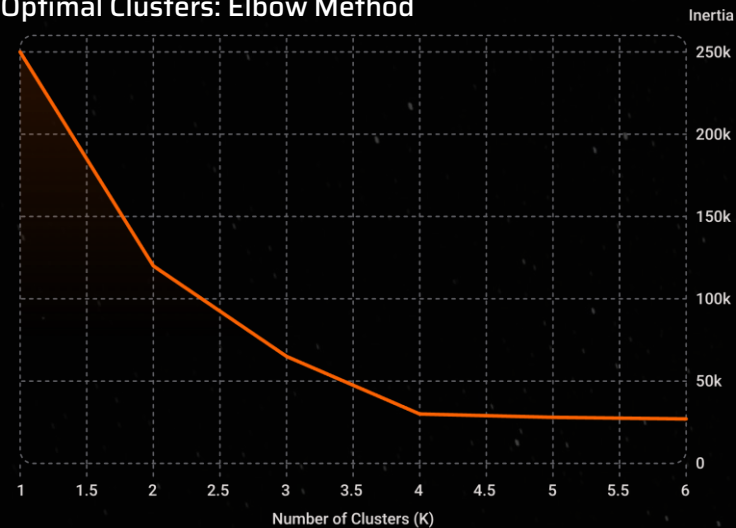


Note: Manual train/test split and manual accuracy calculation used to demonstrate fundamentals (not a black-box).

Results — Visual Highlights

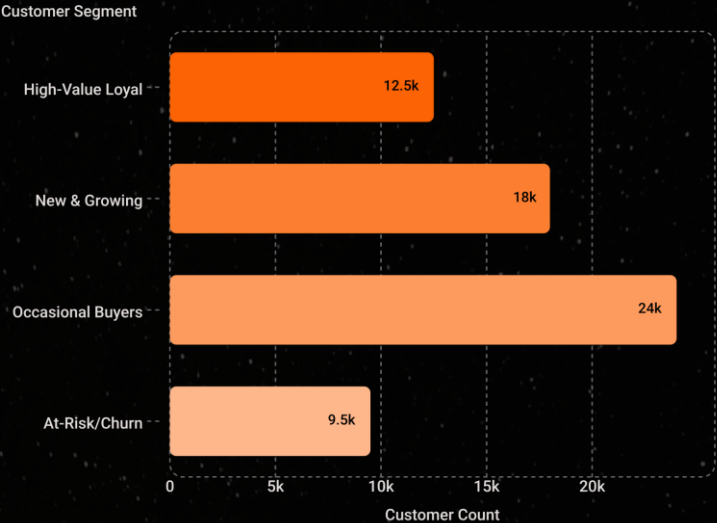
Our analysis reveals actionable insights across customer segmentation and growth prediction.

Optimal Clusters: Elbow Method



The elbow method clearly indicates 4 optimal clusters for customer segmentation, where the decrease in inertia significantly diminishes.

Customer Segments Overview



A breakdown of the four customer segments by their relative sizes, informing targeted marketing strategies.

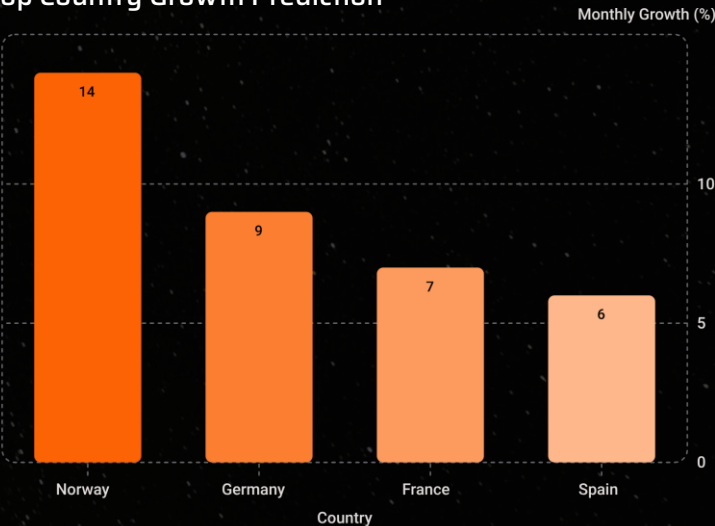
Customer Type Prediction Accuracy

98.6%

Random Forest
Test Accuracy

Our Random Forest model achieved exceptional accuracy in predicting customer segments for new customers.

Top Country Growth Prediction



Norway stands out with an estimated monthly sales growth of ~14%, highlighting key international market potential.

Business Impact & Next Steps

Targeted marketing

Campaigns matched to segment value

Retention priority

Focus on high-value churn risk segments

Instant classification

Predict new-customer type at signup

Expansion decisions

Prioritize countries with positive growth (e.g., Norway)

If more time: add Recency to build RFM, ingest more countries, automate model pipeline and deploy as a live dashboard. Now – I'll switch to the Jupyter notebook to show the implementation.





Thank You!

We appreciate your time and attention today.